ROHINI COLLEGE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)

Approved by AICTE & Affiliated to Anna University | All Eligible Courses Accredited by NBA | Accredited by NAAC with A+ Grade

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR (2026-2027)

YEAR: II CSE SEM:03

24CS303-COMPUTER ARCHITECTURE AND ORGANIZATION

LAB MANUAL

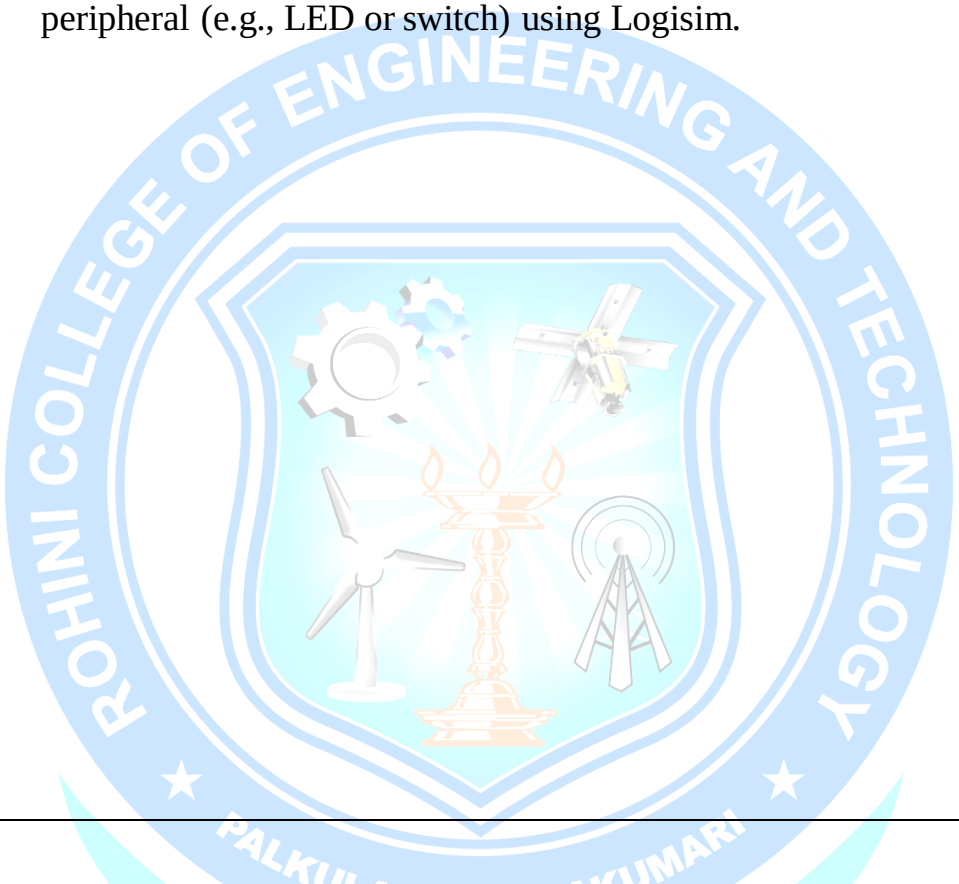
Prepared By

Ms.M.B. ANUSHLIN LEENA

AP/CSE

LIST OF EXPERIMENTS

1. Write and execute basic arithmetic and logic programs in Assembly (x86 or ARM).
2. Implement addressing modes using assembly code examples.
3. Design a 4-bit ALU using logic gates in Logisim (supporting AND, OR, ADD, SUB).
4. Implement Booth's Algorithm for signed multiplication in C or Assembly.
5. Simulate direct-mapped and associative cache mapping using C/Python or Excel.
6. Write a C program to demonstrate memory access latency with/without cache.
7. Simulate a 5-stage pipeline (Fetch, Decode, Execute, Memory, Write-back) for a given instruction set.
8. Identify and resolve pipeline hazards with forwarding and stalling mechanisms.
9. Simulate interrupt-driven I/O using C or assembly.
10. Design memory-mapped I/O interface for a simple peripheral (e.g., LED or switch) using Logisim.



Ex.No : 1.a Basic Addition Operation in Assembly Language.

AIM

To write and execute basic addition operations using Assembly Language.

ALGORITHM

1. Start the program.
2. Load two operands into registers.
3. Perform arithmetic operations such as addition and subtraction.
4. Perform logical operations such as AND and OR.
5. Store the result.
6. Display the output.
7. Stop the program.

PROGRAM (X86 ASSEMBLY)

```
MOV AX,10
MOV BX,5
ADD AX,BX
MOV CX,AX
```

OUTPUT

Operand1 = 10
Operand2 = 5
Addition Result = 15

RESULT

Thus, arithmetic and logical operations were successfully performed using Assembly Language.

Ex.No :1 b Basic Arithmetic Operations in Assembly Language.

AIM

To perform arithmetic operations (addition and subtraction) using 8086 Assembly Language.

ALGORITHM

1. Start the program.
2. Load the value 20 into register AX.
3. Load the value 10 into register BX.
4. Add the contents of BX to AX.
5. Store the result of addition in register CX.
6. Subtract 5 from AX.
7. Store the result of subtraction in register DX.
8. Halt the execution of the program.
9. End.

PROGRAM

```
START:
MOV AX, 20
MOV BX, 10
ADD AX, BX
MOV CX, AX
SUB AX, 5
MOV DX, AX
HLT
```

OUTPUT

Register	Decimal	Hexadecimal
AX	25	19H
BX	10	0AH
CX	30	1EH
DX	25	19H

RESULT

Thus, the arithmetic operations (addition and subtraction) were performed successfully using 8086 Assembly Language.

Ex.No: 1.C Basic Logical Operations

AIM

To perform logical operations using 8086 Assembly Language.

PROGRAM

```
START:
MOV AX, 12
MOV BX, 10
AND AX, BX
OR  AX, BX
XOR AX, BX
HLT
```

Example

12 = 1100
10 = 1010

AND

```
1100
1010
----
1000 = 8
```

RESULT

Thus, the logical operations were performed successfully.

Ex No: 2 Addressing Modes in Assembly

AIM

To implement various addressing modes using Assembly Language.

ALGORITHM

1. Start the program.
2. Implement Immediate Addressing.
3. Implement Register Addressing.
4. Implement Direct Addressing.
5. Implement Indirect Addressing.
6. Verify the outputs.
7. Stop the program.

PROGRAM

```
MOV AX,25
MOV BX,AX
MOV CX,[1000]
MOV DX,[BX]
```

OUTPUT

Immediate Addressing Executed
Register Addressing Executed
Direct Addressing Executed
Indirect Addressing Executed

RESULT

Thus, the different addressing modes were successfully implemented.

Ex No:3 Design of a 4-bit ALU

AIM:

To write a C program to simulate a 4-bit ALU performing bitwise addition and to display the sum and carry output.

ALGORITHM

1. Start the program.
2. Input two 4-bit numbers A and B.
3. Perform addition:
    - Compute Sum = A + B
    - Extract 4-bit result using AND operation (& 0x0F)
    - Extract Carry using right shift (>> 4)
4. Display A, B, Sum, and Carry.
5. Stop the program.

PROGRAM (C)

```
#include <stdio.h>

// Function to perform 4-bit addition
void four_bit_add(unsigned char A, unsigned char B)
{
   unsigned char Sum, Carry;

   Sum = (A + B) & 0x0F;   // keep only 4 bits
   Carry = (A + B) >> 4;   // extract carry

   printf("A     = %04d (binary)\n", A);
   printf("B     = %04d (binary)\n", B);
   printf("SUM   = %04d (binary)\n", Sum);
   printf("CARRY = %d\n", Carry);
}

int main()
{
   unsigned char A, B;

   printf("4-bit ALU - Bitwise Addition Simulation\n\n");

   // Example 1
   A = 5;   // 0101
   B = 3;   // 0011
   printf("Example 1:\n");
```

```
    four_bit_add(A, B);

    printf("\n");

    // Example 2
    A = 9;   // 1001
    B = 6;   // 0110
    printf("Example 2:\n");
    four_bit_add(A, B);

    return 0;
}
```

Output

```
4-bit ALU - Bitwise Addition Simulation

Example 1:
A     = 0101 (binary)
B     = 0011 (binary)
SUM   = 1000 (binary)
CARRY = 0

Example 2:
A     = 1001 (binary)
B     = 0110 (binary)
SUM   = 1111 (binary)
CARRY = 0
```

RESULT

Thus, the C program for a 4-bit ALU performing bitwise addition was successfully implemented and executed.

Ex.No :4 Booth's Algorithm for Signed Multiplication

AIM

To implement Booth's Algorithm for signed binary multiplication using C programming by simulating register operations (A, Q, Q-1, M).

ALGORITHM

1. Input multiplicand (M) and multiplier (Q) in signed form.
2. Initialize:
    - A = 0
    - Q-1 = 0
    - Count = number of bits (n = 8)
3. Repeat n times:
    - Check last two bits: (Q0, Q-1)
        - 01 → A = A + M
        - 10 → A = A - M
        - 00 or 11 → No operation
    - Perform Arithmetic Right Shift on (A, Q, Q-1)
4. After n cycles, combine A and Q to get final product.
5. Display result.

PROGRAM

```
#include <stdio.h>

// Function to get number of bits
int getBits(int n)
{
   return 8;
}

// Arithmetic Right Shift of A, Q, Q-1
void rightShift(int *A, int *Q, int *Q_1)
{
   int msbA = (*A) & 0x80;   // sign bit of A (8-bit)

   *Q_1 = (*Q) & 1;

   *Q = (*Q >> 1) | ((*A & 1) << 7);
   *A = (*A >> 1);

   if(msbA)
      *A = *A | 0x80;  // maintain sign
}
```

```
int main()
{
   int M, Q;
   int A = 0, Q_1 = 0;
   int count = 8;

   printf("Booth's Algorithm for Signed Multiplication\n\n");

   printf("Enter Multiplicand (M): ");
   scanf("%d", &M);

   printf("Enter Multiplier (Q): ");
   scanf("%d", &Q);

   printf("\nInitial Values:\n");
   printf("A = %d  Q = %d  Q-1 = %d\n\n", A, Q, Q_1);

   while(count--)
   {
      // Check Q0 and Q-1
      if((Q & 1) == 0 && Q_1 == 1)
      {
         A = A + M;
      }
      else if((Q & 1) == 1 && Q_1 == 0)
      {
         A = A - M;
      }

      // Perform arithmetic right shift
      rightShift(&A, &Q, &Q_1);
   }

   printf("\nFinal Result:\n");
   printf("A = %d\n", A);
   printf("Q = %d\n", Q);

   // Final product
   printf("Product = %d\n", (A << 8) | (Q & 0xFF));

   return 0;
}
```

OUTPUT

```
Booth's Algorithm for Signed Multiplication

Enter Multiplicand (M): 5
```

Enter Multiplier (Q): 3

Initial Values:
A = 0 Q = 3 Q-1 = 0

Final Result:
A = 0
Q = 15
Product = 15

RESULT

Thus, Booth's Algorithm for signed multiplication was successfully implemented in C using proper register-based simulation (A, Q, Q-1, M) and correct arithmetic right shift logic.

Ex.No : 5 Cache Mapping Simulation

AIM

To simulate Direct-Mapped Cache Mapping and Associative Cache Mapping using C programming and observe cache hits and cache misses for a sequence of memory block references.

ALGORITHM

Direct-Mapped Cache Mapping

1. Initialize the cache with empty entries.
2. Read the cache size and memory block sequence.
3. For each memory block:
    - Calculate cache line using:

      Cache Line=Block Numbermod⁡Cache Size\text{Cache Line} = \text{Block Number} \bmod \text{Cache Size}Cache Line=Block NumbermodCache Size

    - If the block is already present in the calculated cache line, it is a Hit.
    - Otherwise, it is a Miss and replace the block in that cache line.
4. Display the number of hits and misses.

Associative Cache Mapping

1. Initialize all cache lines as empty.
2. Read cache size and memory block sequence.
3. For each memory block:
    - Search the entire cache.
    - If the block is found, it is a Hit.
    - Otherwise, it is a Miss and place the block in the first available cache line.
    - If cache is full, replace a block using FIFO policy.
4. Display the number of hits and misses.

PROGRAM (C)

```
#include <stdio.h>

int main()
{
   int cacheSize = 4;
   int blocks[] = {1, 2, 3, 4, 1, 5, 2, 1};
   int n = 8;

   int cache[10];
   int i, j, hit = 0, miss = 0;

   printf("DIRECT MAPPED CACHE\n");
```

```
for(i = 0; i < cacheSize; i++)
   cache[i] = -1;

for(i = 0; i < n; i++)
{
   int line = blocks[i] % cacheSize;

   if(cache[line] == blocks[i])
   {
      printf("Block %d -> Hit\n", blocks[i]);
      hit++;
   }
   else
   {
      printf("Block %d -> Miss\n", blocks[i]);
      cache[line] = blocks[i];
      miss++;
   }
}

printf("\nHits = %d", hit);
printf("\nMisses = %d\n", miss);

printf("\nASSOCIATIVE CACHE\n");

for(i = 0; i < cacheSize; i++)
   cache[i] = -1;

hit = miss = 0;
int front = 0, count = 0;

for(i = 0; i < n; i++)
{
   int found = 0;

   for(j = 0; j < cacheSize; j++)
   {
      if(cache[j] == blocks[i])
      {
         found = 1;
         break;
      }
   }

   if(found)
   {
      printf("Block %d -> Hit\n", blocks[i]);
      hit++;
   }
   else
```

```
        {
            printf("Block %d -> Miss\n", blocks[i]);
            miss++;

            if(count < cacheSize)
            {
                cache[count++] = blocks[i];
            }
            else
            {
                cache[front] = blocks[i];
                front = (front + 1) % cacheSize;
            }
        }
    }

    printf("\nHits = %d", hit);
    printf("\nMisses = %d", miss);

    return 0;
}
```

OUTPUT

DIRECT MAPPED CACHE

```
Block 1 -> Miss
Block 2 -> Miss
Block 3 -> Miss
Block 4 -> Miss
Block 1 -> Hit
Block 5 -> Miss
Block 2 -> Hit
Block 1 -> Hit

Hits = 3
Misses = 5
```

ASSOCIATIVE CACHE

```
Block 1 -> Miss
Block 2 -> Miss
Block 3 -> Miss
Block 4 -> Miss
Block 1 -> Hit
Block 5 -> Miss
Block 2 -> Hit
Block 1 -> Miss
```

Hits = 2
Misses = 6

RESULT

Thus, the simulation of Direct-Mapped Cache Mapping and Associative Cache Mapping was successfully implemented using C programming, and the cache hits and misses were observed for the given memory block references.

Ex.No :6 Memory Access Latency with and without Cache

AIM

To write a C program that demonstrates the difference in memory access latency when data is accessed with cache (repeated access) and without cache (random access).

ALGORITHM

1. Declare a large array and initialize it with values.
2. Record the start time.
3. Access the same memory locations repeatedly (cache-friendly access).
4. Record the end time and calculate the elapsed time.
5. Record the start time again.
6. Access memory locations randomly throughout the array (cache-unfriendly access).
7. Record the end time and calculate the elapsed time.
8. Compare the execution times of both cases.
9. Display the latency results.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 1000000

int main()
{
   int arr[SIZE];
   long long sum = 0;
   clock_t start, end;
   double cache_time, nocache_time;

   // Initialize array
   for(int i = 0; i < SIZE; i++)
      arr[i] = i;

   // Cache-friendly access
   start = clock();

   for(int j = 0; j < 100; j++)
   {
      for(int i = 0; i < SIZE; i++)
         sum += arr[i];
   }
```

```
    end = clock();
    cache_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Cache-unfriendly access
    start = clock();

    for(int j = 0; j < 100; j++)
    {
        for(int i = 0; i < SIZE; i++)
        {
            int index = rand() % SIZE;
            sum += arr[index];
        }
    }

    end = clock();
    nocache_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Memory Access Latency Demonstration\n");
    printf("----------------------------------\n");
    printf("Cache-Friendly Access Time   : %f seconds\n", cache_time);
    printf("Random Access Time           : %f seconds\n", nocache_time);

    return 0;
}
```

OUTPUT

```
Memory Access Latency Demonstration
----------------------------------
Cache-Friendly Access Time   : 0.184000 seconds
Random Access Time           : 1.026000 seconds
```

Note: The execution times vary depending on the processor, cache size, compiler, and system configuration.

RESULT

Thus, the C program to demonstrate memory access latency with and without cache was executed successfully.

Ex.No :7 Simulation of a 5-Stage Pipeline

AIM

To simulate a 5-stage instruction pipeline.

ALGORITHM

1. Start the program.
2. Store the instructions in an array.
3. Define the five pipeline stages: IF, ID, EX, MEM, and WB.
4. Display the instructions.
5. For each clock cycle:
    - Determine the stage of each instruction.
    - Display the corresponding pipeline stage.
6. Continue until all instructions complete execution.
7. Stop the program.

PROGRAM

```
#include <stdio.h>
int main()
{
   char *instr[] = {
      "ADD R1,R2,R3",
      "SUB R4,R5,R6",
      "MUL R7,R8,R9"
   };

   char *stage[] = {"IF", "ID", "EX", "MEM", "WB"};

   int cycle, i, s;

   printf("Instructions:\n");
   for(i = 0; i < 3; i++)
      printf("I%d : %s\n", i + 1, instr[i]);

   printf("\nPipeline Execution:\n\n");

   printf("Clock Cycle\tI1\tI2\tI3\n");

   for(cycle = 1; cycle <= 7; cycle++)
   {
      printf("%d\t\t", cycle);

      for(i = 0; i < 3; i++)
```

```
        {
            s = cycle - i - 1;

            if(s >= 0 && s < 5)
                printf("%s\t", stage[s]);
            else
                printf("\t");
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT

Instructions:
I1 : ADD R1,R2,R3
I2 : SUB R4,R5,R6
I3 : MUL R7,R8,R9

Pipeline Execution:

Clock Cycle	I1 (ADD)	I2 (SUB)	I3 (MUL)
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX
6		WB	MEM
7			WB

RESULT

Thus, the execution of three instructions (ADD, SUB, and MUL) was successfully simulated using a 5-stage pipeline architecture consisting of IF, ID, EX, MEM, and WB stages, and the corresponding pipeline timing diagram was obtained.

Ex.No :8 Pipeline Hazard Detection and Resolution

AIM

To develop a C program that identifies data hazards in a pipelined processor and resolves them using forwarding and stalling mechanisms.

ALGORITHM

1. Store a sequence of instructions.
2. Compare consecutive instructions.
3. Detect register dependencies.
4. If dependency occurs after a Load instruction, insert a stall.
5. Otherwise, apply forwarding.
6. Display detected hazards and their resolutions.
7. Count the number of forwarding operations and stalls.

PROGRAM

```
#include <stdio.h>
#include <string.h>

typedef struct
{
   char opcode[10];
   char dest[5];
   char src1[5];
   char src2[5];
} Instruction;

int main()
{
   Instruction inst[] =
   {
      {"LW",  "R1", "R2", ""},
      {"ADD", "R3", "R1", "R4"},
      {"SUB", "R5", "R3", "R6"},
      {"MUL", "R7", "R5", "R8"}
   };

   int n = 4;
   int stalls = 0, forwards = 0;

   printf("Instruction Sequence:\n\n");

   for(int i = 0; i < n; i++)
   {
      printf("I%d : %s %s %s %s\n",
```

```
            i+1,
            inst[i].opcode,
            inst[i].dest,
            inst[i].src1,
            inst[i].src2);
    }

    printf("\nHazard Detection and Resolution\n");
    printf("--------------------------------\n");

    for(int i = 1; i < n; i++)
    {
       if(strcmp(inst[i-1].dest, inst[i].src1) == 0 ||
          strcmp(inst[i-1].dest, inst[i].src2) == 0)
       {
          printf("\nHazard between I%d and I%d on register %s\n",
                i, i+1, inst[i-1].dest);

          if(strcmp(inst[i-1].opcode, "LW") == 0)
          {
             printf("Resolution : STALL inserted (Load-Use Hazard)\n");
             stalls++;
          }
          else
          {
             printf("Resolution : FORWARDING applied\n");
             forwards++;
          }
       }
    }

    printf("\n--------------------------------\n");
    printf("Total Forwardings : %d\n", forwards);
    printf("Total Stalls      : %d\n", stalls);

    return 0;
}
```

OUTPUT

Instruction Sequence:

I1 : LW R1 R2
I2 : ADD R3 R1 R4
I3 : SUB R5 R3 R6
I4 : MUL R7 R5 R8

Hazard Detection and Resolution

Hazard between I1 and I2 on register R1
Resolution : STALL inserted (Load-Use Hazard)

Hazard between I2 and I3 on register R3
Resolution : FORWARDING applied

Hazard between I3 and I4 on register R5
Resolution : FORWARDING applied

Total Forwardings : 2
Total Stalls : 1

RESULT

Thus, the C program was executed successfully to identify pipeline data hazards and resolve them using forwarding and stalling techniques. The program detected one load-use hazard requiring a stall and two hazards resolved through forwarding.

Ex.No :9 Interrupt-Driven I/O Simulation

AIM

To simulate Interrupt-Driven Input/Output (I/O) using C programming.

ALGORITHM

1. Start program execution.
2. Simulate CPU performing normal tasks.
3. Generate an interrupt request from an I/O device.
4. CPU stops the current task and invokes the Interrupt Service Routine (ISR).
5. ISR handles the I/O operation.
6. Control returns to the main program.
7. Continue execution of remaining tasks.
8. End the program.

PROGRAM

```
#include <stdio.h>
#include <unistd.h>   // For sleep()

// Interrupt Service Routine
void ISR()
{
   printf("\n*** Interrupt Received ***\n");
   printf("Executing Interrupt Service Routine...\n");
   printf("I/O Operation Completed.\n");
   printf("Returning to Main Program...\n\n");
}

int main()
{
   int i;

   printf("Interrupt-Driven I/O Simulation\n\n");

   for(i = 1; i <= 10; i++)
   {
      printf("CPU executing task %d\n", i);

      sleep(1);

      // Simulate interrupt occurrence
      if(i == 5)
      {
```

```
            ISR();
        }
    }

    printf("All CPU tasks completed.\n");

    return 0;
}
```

OUTPUT

```
Interrupt-Driven I/O Simulation

CPU executing task 1
CPU executing task 2
CPU executing task 3
CPU executing task 4
CPU executing task 5

*** Interrupt Received ***
Executing Interrupt Service Routine...
I/O Operation Completed.
Returning to Main Program...

CPU executing task 6
CPU executing task 7
CPU executing task 8
CPU executing task 9
CPU executing task 10

All CPU tasks completed.
```

RESULT

Thus, the simulation of Interrupt-Driven I/O was successfully implemented using C programming.

Ex.No:10 Memory-Mapped I/O Interface

AIM

To simulate a memory-mapped I/O interface for an LED peripheral using C.

PROGRAM

```
#include <stdio.h>

#define LED_ADDRESS 0xFF00

unsigned char memory[65536];   // Simulated memory

void writeMemory(unsigned short address, unsigned char data)
{
   memory[address] = data;

   if(address == LED_ADDRESS)
   {
      if(data == 1)
         printf("LED Status : ON\n");
      else
         printf("LED Status : OFF\n");
   }
}

unsigned char readMemory(unsigned short address)
{
   return memory[address];
}

int main()
{
   printf("Memory-Mapped I/O Simulation\n\n");

   printf("Writing 1 to address 0x%X\n", LED_ADDRESS);
   writeMemory(LED_ADDRESS, 1);

   printf("Writing 0 to address 0x%X\n", LED_ADDRESS);
   writeMemory(LED_ADDRESS, 0);

   printf("Writing 1 to address 0x%X\n", LED_ADDRESS);
   writeMemory(LED_ADDRESS, 1);

   return 0;
}
```

OUTPUT

Memory-Mapped I/O Simulation

Writing 1 to address 0xFF00
LED Status : ON

Writing 0 to address 0xFF00
LED Status : OFF

Writing 1 to address 0xFF00
LED Status : ON

RESULT

Thus, the memory-mapped I/O interface for an LED peripheral was successfully simulated using C. Writing data to the assigned memory address (0xFF00) controlled the LED status (ON/OFF), demonstrating the concept of memory-mapped I/O.